

Matriks Migrasi & Benchmarking Build Tools — Modul 8

Proyek: MaternalGuard | **Kelas:** TI-A | **Anggota:** Ellysa Nur Muzayyana (V3925043), Hertyningtias Tata Meirzha (V3925044)

1. Konteks & Keterbatasan Migrasi

MaternalGuard dibangun di atas **Next.js 14 (App Router)**, memakai Server Components, Server Actions, Route Handler, dan middleware autentikasi. Next.js memiliki compiler & bundler sendiri (SWC + Webpack) dan **tidak dapat diganti dengan Vite** tanpa membongkar seluruh fitur di atas — Vite dirancang untuk SPA murni, bukan framework full-stack.

Karena itu, migrasi dilakukan dalam dua bagian yang saling melengkapi, bukan migrasi total:

Bagian	Pendekatan
Aplikasi utama (Next.js)	Tetap Next.js , tapi ditambah Biome (linter/formatter Rust) & Strict TypeScript (noUncheckedIndexedAccess) — dua dari tiga tuntutan modul yang independen dari bundler
Build Tools Lab (baru)	Vite + React + TS + Biome dibangun sebagai proyek terpisah (build-tools-lab/uns-fastbuild-app) mengikuti persis Langkah 1–5 modul, untuk memenuhi tuntutan vite.config.ts, manualChunks, dan benchmarking dev-server secara harfiah

2. Konfigurasi yang Diterapkan

- **vite.config.ts:** path alias @/ → ./src, server.port tetap, build.rollupOptions.output.manualChunks memisahkan react/react-dom ke chunk vendor terpisah dari kode aplikasi.
- **biome.json:** linter + formatter (line width 100, indent 2 spasi), diterapkan di **kedua** proyek (Next.js utama & Vite lab).
- **tsconfig:** strict: true + noUncheckedIndexedAccess: true di kedua proyek. Pengetatan ini **menemukan 1 bug nyata** (akses index array yang berpotensi undefined di FollowUpWidget.tsx) yang langsung diperbaiki.

3. Hasil Benchmark (Diukur Langsung)

Metrik	Next.js 14 (Webpack/SWC)	Vite 8 (Esbuild + Rolldown)	Selisih
Dev server cold start (Ready)	1.318 s	0.205 s	Vite ±6,4× lebih cepat
Production build	34,8 s	3,5 s total (238 ms)	Vite jauh lebih cepat

Metrik	Next.js 14 (Webpack/SWC)	Vite 8 (Esbuild + Rolldown)	Selisih
(clean, tanpa cache)		proses bundling murni)	
Quality Gate (biome check)	0 error, 5 warning (! important, disengaja)	0 error, 0 warning	Keduanya lolos
Type-check (tsc -- noEmit, strict)	0 error	0 error	Keduanya lolos

Diukur di lingkungan sandbox Linux (container), bukan laptop mahasiswa — rasio kecepatan relatif tetap representatif, namun angka absolut bisa berbeda di perangkat lain. Mahasiswa disarankan mengukur ulang secara lokal untuk laporan yang lebih presisi.

Catatan kejujuran metodologi: kedua aplikasi berbeda skala (Next.js: 8 route, Server Components, middleware auth, Route Handler; Vite lab: 1 halaman SPA). Karena itu **ukuran bundle sengaja tidak dibandingkan langsung** — yang dibandingkan secara adil hanyalah waktu cold-start dan build tool itu sendiri, bukan total output aplikasi.

4. Pemetaan 7 Poin Persyaratan Modul

#	Persyaratan	Status	Lokasi
1	Konfigurasi Vite & Path Aliasing	✓	build-tools- lab/uns- fastbuild-app/ vite.config.ts
2	Integrasi Biome (Rust)	✓	biome.json (root & lab)
3	Strict TypeScript + noUncheckedIndex edAccess	✓	tsconfig.json (root), tsconfig.app.js on (lab)
4	Code Splitting (manualChunks)	✓	Terverifikasi: vendor-*.js terpisah dari index- *.js pada dist/
5	Pengujian HMR & Dev Server	✓	Tabel benchmark di atas; HMR Vite diuji manual via tombol interaktif di dashboard lab
6	Quality Gate (biome check)	✓	Exit code 0 di kedua proyek
7	Dokumen Matriks & Benchmarking	✓	Dokumen ini

5. Kesimpulan

Next.js tidak dimigrasikan ke Vite karena keduanya menyasar kebutuhan arsitektur yang berbeda (full-stack framework vs. SPA build tool). Sebagai gantinya, prinsip inti modul — toolchain berbasis Rust (Biome), Strict TypeScript, dan build tool ber-performa tinggi (Vite + Esbuild + Rolldown) — diterapkan secara nyata dan terukur: Biome + strict TS pada aplikasi utama, dan proyek Vite penuh sebagai laboratorium benchmarking yang membuktikan klaim kecepatan modul (cold start $\sim 6\times$ lebih cepat, build jauh lebih ringan) dengan angka asli, bukan simulasi.